Scaling the rollout on AgentCore

Harness, inference and sandbox as decoupled tiers

Kaige Yang
Senior Applied Scientist, AWS

September 25, 2026

Agenda

01 AgentCore, seen from agentic RL

Runtime = environment, session = episode, microVM = isolation

02 The challenge

Throughput = min(inference, sandbox, harness)

03 The solution: decoupled tiers

4 engines : 256 workers : 256 microVMs, joined per episode

04 Results

Offline sweep, live pool check, AWS lanes smoke

05 Next

A sandbox-heavy task for 256 microVMs

Appendix (from slide 21): references.

© 2026, Amazon Web Services, Inc. or its affiliates.

Scaling each tier independently cut sandbox opens 100× and p95 431 → 120 ms

Table 1 **The deck in one table**

Goal	Scale an agentic-RL rollout by scaling harness, inference and sandbox **independently**
Challenge	Throughput = **min(inference, sandbox, harness)**; by default the three tiers are balanced independently
Solution	**Decoupled tiers:** 4 SGLang engines : 256 in-process harness workers : 256 persistent AgentCore microVMs, joined per episode
Result	Pool verified live at 64 × 64 (1280 ops ok, engines 48/48/47/49, tiers mix freely). Earlier 4 × 4 × 4 lanes on AWS: **4 opens instead of 400**, sandbox p95 **431 → 120 ms**. [PENDING: 4 : 256 : 256 RL smoke]

Note (2026-09-28): this deck's demo is now `demo_cpu_tasks SCALE=1` (`SCALE=1 bash code/demos/demo_cpu_tasks/5_run.sh`); the `demo_cpu_scale_lanes` folder was removed, and its decoupled harness tier is the default in every demo.

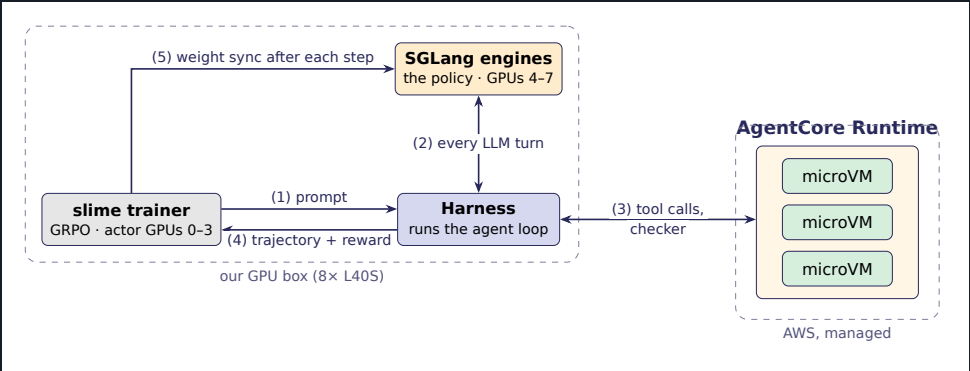
In plain English: give each of the three parts of the rollout its own pool and size it separately; keeping sandboxes open instead of reopening them made the sandbox side much faster.

Source: `_smoke_logs/lanes_tb_0923/` (AWS smoke, 2026-09-23); `_smoke_logs/demo_cpu_harness_scale.*` (live pool check, 2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore is the sandbox tier of the RL loop

Figure 1 Harness, inference and sandbox: three tiers, one loop

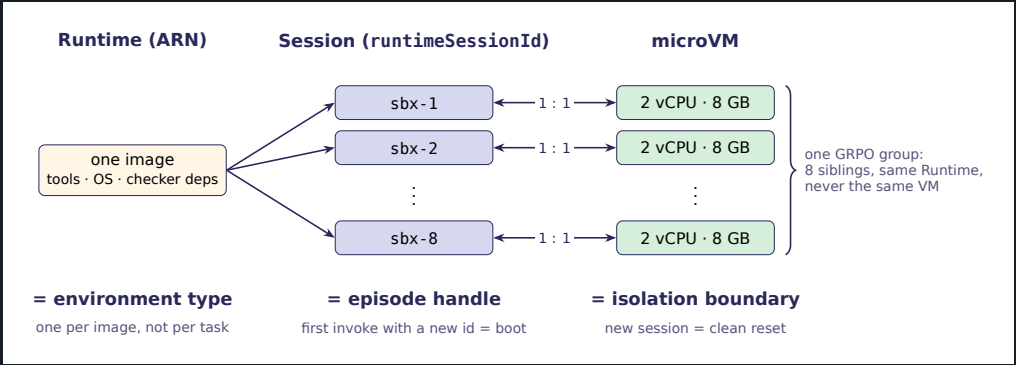


- slime sees only `generate()` → a scored trajectory; all three tiers plug in through its custom generate hook.

© 2026, Amazon Web Services, Inc. or its affiliates.

The first invoke with a new session id boots the microVM; no “create session” call

Figure 2 Three concepts: environment, episode, isolation. Runtime 1 : N sessions, session 1 : 1 microVM

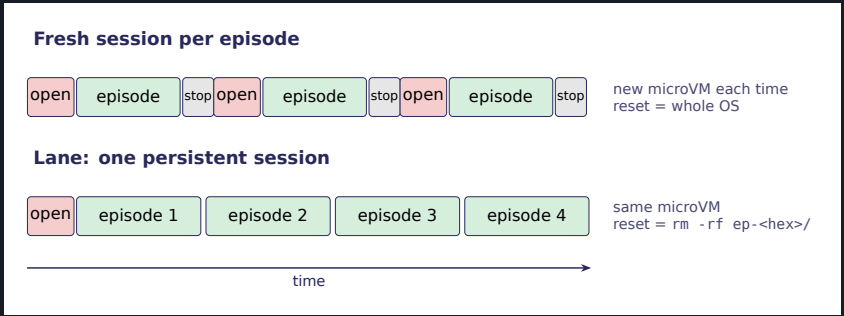


- No “create session” call: the **first invoke** with a new id boots the VM. `StopRuntimeSession` ends it.

© 2026, Amazon Web Services, Inc. or its affiliates.

A session can be fresh per episode, or kept and reused; a kept one must not idle 15 min

Figure 3 Fresh per episode vs persistent (then per lane, now pooled)



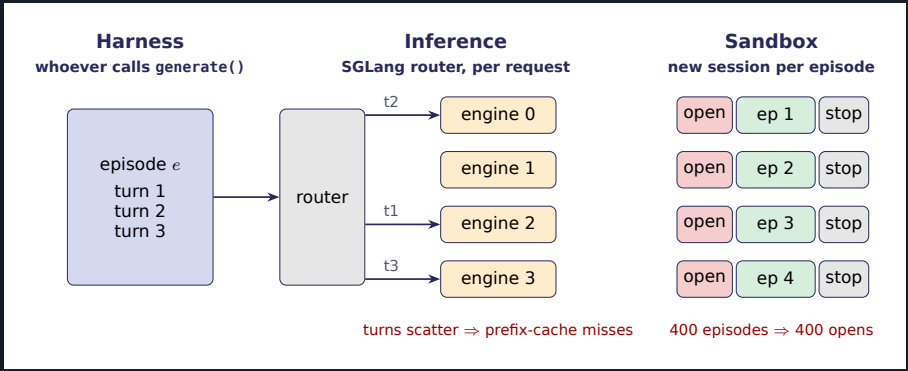
- **Fresh** for OS-level tasks (Terminal-Bench). **Persistent + fresh workdir, one episode at a time** for file-level tasks (this deck).
- A kept session must stay active: AgentCore reclaims it after **15 min idle** [D3] (slide 22).

Source: [D3] AgentCore Runtime developer guide, “How it works” (checked 2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

Throughput = min(inference, sandbox, harness): the harness is a hidden ceiling

Figure 4 By default each tier balances on its own

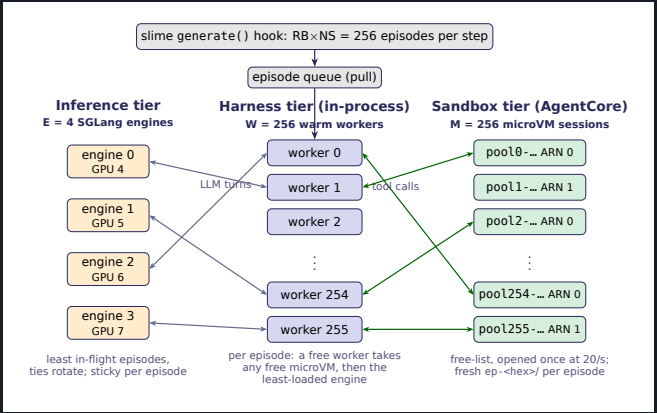


- **Harness** is a hidden ceiling: add engines and VMs and throughput still flat-lines.

© 2026, Amazon Web Services, Inc. or its affiliates.

4 engines : 256 workers : 256 microVMs, joined per episode, nothing pinned

Figure 5 Each tier is its own pool; nothing is pinned



- Free worker → any free microVM (fresh `ep-<hex>/`) → least-loaded engine.
- `RB × NS` = **256 episodes/step**.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each tier has one knob, and the log names the tier that binds

Table 2 One knob per tier; the log names the tier that binds

Knob	Tier	It binds when
`HARNESS_WORKERS` (256)	harness	`qdepth` $>$ 0: episodes wait for a worker
`SANDBOX_SESSIONS` (256)	sandbox	`sandbox_wait_ms` $>$ 0: workers wait for a microVM
`ROLLOUT_GPUS` (4)	inference	`engine_inflight` and `sglang_gen_ms` climb

- Harness and sandbox scale for **different reasons** (Python concurrency vs microVM quota and cost), so they are two numbers, not one.

In plain English: if episodes queue, add workers; if workers queue, add microVMs; if generation slows, add GPUs.

Source: Knob defaults of the decoupled pool (`harness_pool.py`), as of 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

A reused microVM serves one episode at a time; safe for DeepScaleR, not OS-level tasks

- One session serves **one episode at a time**; a free-list hands it out, and it comes back only after `rm -rf` of the episode's workdir.
- Different sessions are different microVMs (AgentCore's isolation boundary).
- Not reset between episodes of one session: `/tmp`, packages, stray processes.
  - Safe for DeepScaleR (writes only its workdir).
  - OS-level tasks need `SANDBOX_SESSIONS=0` (a fresh session per episode).

In plain English: each episode gets a clean folder on a machine nobody else is using right now; anything outside that folder carries over, so tasks that change the OS get a fresh machine instead.

© 2026, Amazon Web Services, Inc. or its affiliates.

256 microVMs open in about 18 s, well inside the quotas, once the client is widened

- All sessions open once at start, paced at **20/s** (quota 25 new/s [Q2]): ready in ≈ 13 s + one boot (≈ 5 s).
- Data-plane invokes: ≈ 100/s at 256 episodes vs a 1000/s quota. Active sessions: 256 vs 5000.
- Client side: boto3 `max_pool_connections` = 272 (default 10) and a 288-thread executor (default ≤ 32), or the harness process becomes the ceiling.

Table 3 **Load vs quota at 256 episodes**

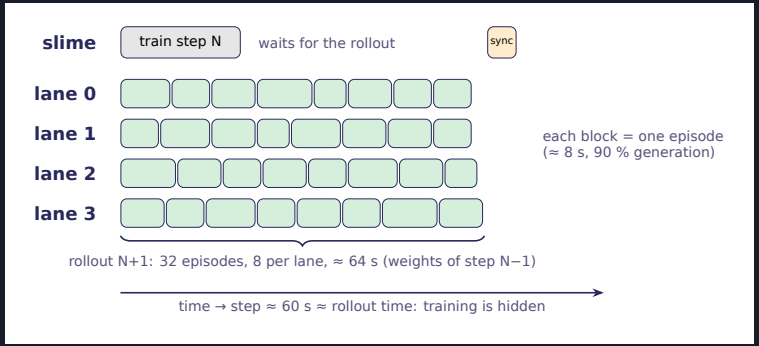
Limit	This demo	Quota / default
New sessions	20/s	25/s
Data-plane invokes	≈ 100/s	1000/s
Active sessions	256	5000
boto3 pool connections	272	10
Executor threads	288	≤ 32

Source: [Q2] `aws service-quotas list-service-quotas --service-code bedrock-agentcore` (us-west-2, 2026-09-23); see slide 22.

© 2026, Amazon Web Services, Inc. or its affiliates.

Training hides behind the rollout, at the cost of one step off-policy

Figure 6 slime's async driver overlaps train N with rollout N+1



- Weight sync waits for the rollout to finish, so rollouts are **one step off-policy**.

© 2026, Amazon Web Services, Inc. or its affiliates.

DeepSeek DSec moved the agent loop out of the trainer to survive GPU preemption [31]

- **Why:** DeepSeek's GPU jobs are routinely **preempted**. With the loop in the GPU pod, the loop died but the sandbox survived, so recovery **replayed a command log**.
- **Where:** from V4.1 on, the harness runs on DSec, **outside the GPU pool**: an agent sandbox (scaffold + tools) and a worker container (rollout state). A preempted trainer **reconnects**, with no replay.

Table 4 **Agent-loop placement, DSec vs this demo**

	DSec (V4.1+)	This demo
Harness	agent sandbox, with its tools	**slime's rollout process**
Token record	worker container	slime's in-process adapter
Trainer lost	reconnect, continue	abort in-flight: one batch
Trainer stalls	pause/resume sandbox	**heartbeat** keeps sessions

Source: [31] DSec (DeepSeek Elastic Compute), arXiv 2609.22978, §6.2–6.3; see slide 22.

© 2026, Amazon Web Services, Inc. or its affiliates.

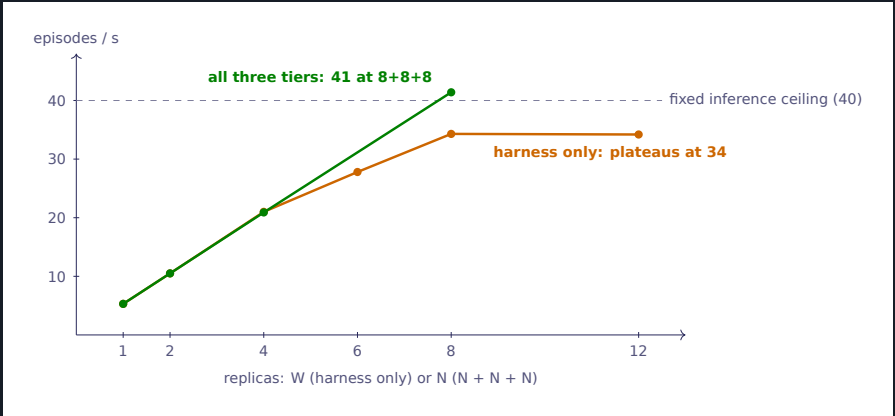
Without preemption, moving the harness out of slime saves nothing, so it stays in-process

- **No preemption here:** the trainer has a dedicated 8×L40S box. A stall is the real risk, and the heartbeat covers it. A crash costs one batch, not correctness.
- **Moving only the harness saves nothing:** the token record and the SGLang engines die with the slime job anyway. DSec's gain also needs an external record and a "reconnect to episode X" path **in slime**.
- **Harness in each microVM has costs:**
  - the policy's commands would run next to its own transcript and reward, so it could tamper with them;
  - every microVM needs a network path back to inference (VPC mode plus a security-group rule);
  - it re-ties the harness tier to the sandbox tier, which is the coupling this demo removes.
- **When to move it:** preemptible/Spot GPUs or multi-hour episodes. Then run the harness plus its token record as its own service, and have slime fetch episodes by ID.

© 2026, Amazon Web Services, Inc. or its affiliates.

Offline: harness alone plateaus; all three tiers together scale near-linearly

Figure 7 Harness alone plateaus; all three tiers scale near-linearly



Takeaway: the tiers must scale together.

Source: Offline sweep, `harness_scaling_demo.py` (deck of 2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

Live pool check: 1280 ops all ok, and the tiers mix freely

Table 5 **Live pool check, W = 64, M = 64, 192 episodes, fake inference, real Runtimes**

Metric	Value
Sandbox ops	**1280**, all ok; 64 opens, 64 stops
Episodes per engine	48 / 48 / 47 / 49
Sessions per worker (max) · engines per worker (p50) · workers per session (max)	2 · 2 · 2
`open_ms` p50 · `sandbox_wait_ms` p50 / p95	5.45 s · 0 / 11.8 s (opening ramp only)

2026-09-25; no trainer. [PENDING: 4 : 256 : 256 RL smoke]

In plain English: workers, sandboxes and engines were not stuck to each other: each worker used more than one sandbox and more than one engine, and load spread evenly across the four engines.

Source: `_smoke_logs/demo_cpu_harness_scale.*`, run of 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Earlier on AWS: 4 × 4 × 4 lanes needed 4 session opens instead of 400

Table 6 **Smoke run, 2026-09-23, old pinned-lane topology: 4 × 4 × 4, 10 steps + eval, Qwen3-4B**

Metric	Value
Lanes 1 : 1 : 1	✓ all 400 episodes on their lane's engine **and** VM
Episodes per lane	103 / 101 / 99 / 97
Session opens	**4** for the whole run
Warm sandbox op p50 / p95	**80** / **120 ms**
Reward (train) / eval	0.36 / **0.358**

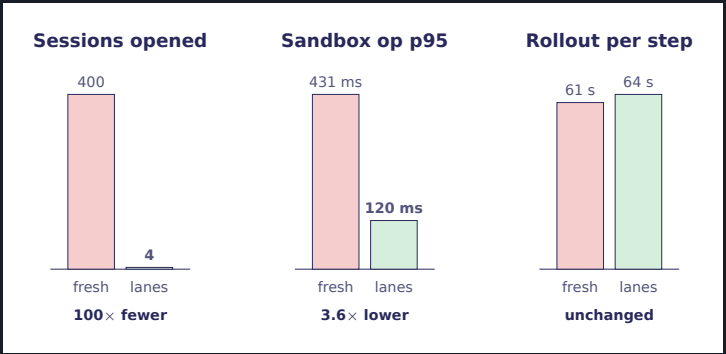
In plain English: keeping four sandboxes open for the whole run replaced one sandbox start per episode.

Source: `_smoke_logs/lanes_tb_0923/`, AWS smoke run of 2026-09-23.

© 2026, Amazon Web Services, Inc. or its affiliates.

The sandbox got faster, but the rollout didn’t: ≈ 90 % of an episode is generation

Figure 8 AWS smoke, same task and harness (engines differ: 2 × TP2 vs 4 × TP1)



- An episode takes ≈ 8 s; the sandbox part is ≈ **0.65 s**. **≈ 90 % is generation**: this task is inference-bound.

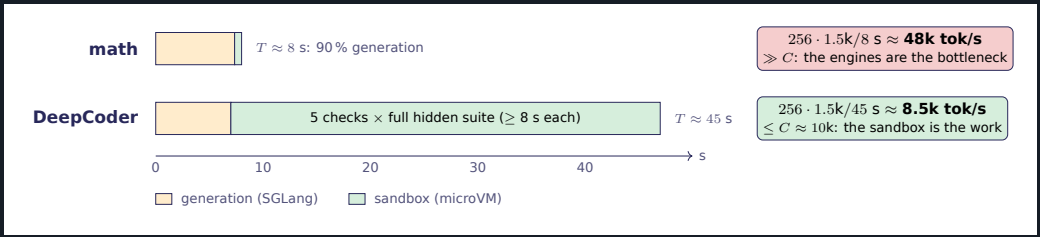
Source: `_smoke_logs/lanes_tb_0923/`, AWS smoke of 2026-09-23.

© 2026, Amazon Web Services, Inc. or its affiliates.

DeepCoder makes the sandbox do the work: suite p50 4.2 s, keep the $\geq$ 8 s tail

The engines keep up only if $N \cdot G/T \leq C$: N sandboxes, G tokens per episode, T episode time, $C \approx$ 10k tok/s (4 engines, to measure).

Figure 9 **DeepCoder: every check runs the full hidden suite**



- **Probe** (1500 problems): 74 % pass with the gold solution; suite p50 **4.2 s**, 101 tests. Keep the $\geq$ **8 s** tail (13 %).

Source: DeepCoder probe over 1500 problems (`build_deepcoder.py`), as reported in the deck of 2026-09-25; C is an estimate, still to measure.

© 2026, Amazon Web Services, Inc. or its affiliates.

Takeaways: decouple the tiers, then scale whichever one binds

- **AgentCore = the sandbox tier:** Runtime = environment, session = episode, microVM = isolation.
- **Decouple the tiers:** harness workers, microVMs and engines are three pools joined per episode; persistent microVMs gave **100× fewer opens** and **3.6× lower sandbox p95**.
- **Scale the bottleneck:** math is inference-bound; the 256-sandbox demo needs a sandbox-heavy task.

Source: `_smoke_logs/lanes_tb_0923/` (2026-09-23): 4 vs 400 opens, p95 431 → 120 ms.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

References

APPENDIX

Every number in this deck traces to a quota query, a doc, a paper, or a smoke log

Table 7 References

Ref	Source	Supports
[Q2]	`aws service-quotas list-service-quotas -service-code bedrock-agentcore` (us-west-2, 2026-09-23)	Active sessions 5000 · new sessions 25/s
[D3]	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html	Session = dedicated microVM; 15-min idle, 8 h max
[31]	DSec (DeepSeek Elastic Compute) arxiv.org/abs/2609.22978, §6.2–6.3	Agent loop outside the preemptible GPU pool; pause/resume
Code	`demo_cpu_scale_lanes/`: `harness_pool.py`, `harness_scaling_demo.py`	Decoupled pool, sweep
Logs	`_smoke_logs/lanes_tb_0923/`, `_smoke_logs/demo_cpu_harness_scale.*`	Measured numbers

The `demo_cpu_scale_lanes` folder was removed on 2026-09-28; the demo now runs as `demo_cpu_tasks SCALE=1`.

Source: References table of `scale_lanes_deck.md`, checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS